Failure (a crashing tool, and a bad description) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `tool_selection_practice.py` (Failure section): a crashing tool and a vague description are both real production bugs you will cause yourself at least once — better to see them here, on purpose, where nothing's at stake. **If not:** the Build Task's "one tool designed to sometimes fail" would be the first time you ever turned a raised exception into a clean, model-readable error string.

Solution

```
# tool_selection_practice.py — Failure section
from langchain_core.tools import BaseTool, tool
from langchain_openai import ChatOpenAI

# --- Half 1: a crashing tool, made safe, and the error sent back ---

@tool
def flaky_divide(a: int, b: int) -> str:
    """Divide a by b. Returns an error message string if b is zero."""
    try:
        result = a / b
        return str(result)
    except ZeroDivisionError as e:
        return f"Error: {e}"

def demonstrate_safe_failure() -> None:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([flaky_divide])
    # how: the same growing message list as Doc04's memory exercise
    messages = [{"role": "user", "content": "Use the tool to divide 10 by 0"}]
    response = model_with_tools.invoke(messages)
    if not response.tool_calls:
        print(f"No tool called: {response.content}")
        return

    # how: the model's tool request goes into the history first
```

```
    messages.append(response)
    for call in response.tool_calls:
        result = flaky_divide.invoke(call["args"])
        print(f"Tool call {call['args']} -> {result}")
        # why: tool_call_id ties this result to the exact request, so the
        # model knows which of its calls this answer belongs to
        messages.append(
            {"role": "tool", "content": result, "tool_call_id": call["id"]}
        )

    # why: a second call, now with the error in the history — this is
    # the model really reacting to the tool's result
    final = model_with_tools.invoke(messages)
    print(f"Model's reaction to the error: {final.content}")

# --- Half 2: a vague description, sharpened ---
# how: @tool("get_weather") gives both tools the same name, so the
# description is the only thing that differs between the two trials

@tool("get_weather")
def vague_weather(city: str) -> str:
    """gets data"""
    return f"Sunny in {city}"

@tool("get_weather")
def precise_weather(city: str) -> str:
    """Get the current weather (temperature, conditions) for a named city."""
    return f"Sunny in {city}"

# how: BaseTool is the type every @tool function has — so this hint
# says "weather_tool is a tool", the same way `prompt: str` says text
def count_tool_calls(weather_tool: BaseTool, prompt: str, runs: int = 5) -> int:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([weather_tool])
    called = 0
    for run_number in range(runs):
        response = model_with_tools.invoke(prompt)
        if response.tool_calls:
            called = called + 1
    return called

def demonstrate_description_fix() -> None:
    prompt = "what's the weather like"
    vague_hits = count_tool_calls(vague_weather, prompt)
    precise_hits = count_tool_calls(precise_weather, prompt)
    print(f"Vague description: called {vague_hits}/5 times")
    print(f"Precise description: called {precise_hits}/5 times")
```

```
if __name__ == "__main__":
    demonstrate_safe_failure()
    print()
    demonstrate_description_fix()
```

Expected output (example):

```
Tool call {'a': 10, 'b': 0} -> Error: division by zero
Model's reaction to the error: Dividing by zero isn't possible.

Vague description: called 1/5 times
Precise description: called 5/5 times
```